

Manuale Tecnico

Niccolò Jacopo Alt, Alessia Gerti, Mateo Soldo, Davide Vignati

20/07/2026

Indice

1	Introduzione	3
2	Installazione e configurazione dell'ambiente	3
2.1	Installazione dei componenti software	3
2.1.1	Java Development Kit (JDK 25 LTS)	3
2.2	Apache Maven	5
2.3	PostgreSQL RDBMS	5
2.3.1	Installazione	5
2.4	Gestione della Build e Comandi Maven	6
2.5	Ambienti di Sviluppo Integrati (IDE)	6
3	Progettazione e architettura del software (UML)	7
3.1	Modello Architetturale	7
3.2	Diagramma delle classi e organizzazione dei package	7
3.2.1	Package <code>cinemax.common</code>	8
3.2.2	Package <code>cinemax.client</code>	9
3.2.3	Package <code>cinemax.server</code>	9
3.3	Design patterns utilizzati	9
3.3.1	Singleton Pattern (Creazionale)	9
3.3.2	Data Access Object - DAO Pattern (Strutturale)	10
3.3.3	Factory Pattern (Creazionale)	10
3.3.4	Observer Pattern (Comportamentale)	10
3.4	Processo transazionale e concorrente di prenotazione posto	10
3.4.1	Fase di autenticazione e login	11
4	Progettazione della base di dati ER Relazionale	12
4.1	Analisi dei requisiti	12
4.2	Modello concettuale (Diagramma ER)	12
4.2.1	Specifiche delle relazioni e cardinalità	13
4.3	Vincoli in linguaggio naturale	13
4.4	Ristrutturazione dello schema ER	14
4.4.1	Eliminazione di attributi composti o multivalore	14
4.4.2	Analisi delle ridondanze	14
4.5	Modello Logico Relazionale	14
4.6	Analisi delle Dipendenze Funzionali e Normalizzazione	15
4.7	Script DDL di Creazione del Database (PostgreSQL)	15
5	Sitografia e bibliografia	16

1 Introduzione

Il manuale tecnico nasce con l'obiettivo di descrivere nel dettaglio le scelte progetturali, architetture e tecnologiche adottate nello sviluppo del sistema **CineMax**.

La documentazione si focalizza sull'estensione dell'applicazione verso un'architettura *Client* e *Server* multithread e sull'interrogazione di una base di dati relazionale per la persistenza delle informazioni.

All'interno della documentazione viene fornita una panoramica esaustiva sugli strumenti, sulle librerie esterne e sui driver di connessione (JDBC) utilizzati. Vengono illustrati i principi che hanno guidato la realizzazione del software con riferimento ai design pattern orientati agli oggetti, alle strutture dati impiegate e alla gestione degli accessi concorrenti alle risorse condivise.

Un ampio spazio è dedicato alla formalizzazione dei modelli grafici, sia a livello logico-funzionale del codice tramite diagrammi UML (*Unified Modeling Language*), sia a livello della persistenza dei dati attraverso la modellazione concettuale ER (*Entity-Relationship*) e la derivazione dello schema relazionale normalizzato.

Infine, il manuale tecnico serve per chiarire la logica architetture e transazionale alla base del progetto, facilitandone così la comprensione, la manutenibilità e l'eventuale ripresa futura, evidenziando le potenzialità di un approccio modulare e scalabile che permette all'applicazione di crescere ed evolversi senza perdere coerenza interna.

2 Installazione e configurazione dell'ambiente

Per poter configurare l'ambiente di sviluppo, compilare ed eseguire il sistema **CineMax** è necessario installare e configurare i componenti software descritti di seguito. Le procedure fanno riferimento a un'installazione centralizzata su una singola macchina di sviluppo (host locale), sulla quale risiederanno sia l'ambiente di runtime che la base di dati.

2.1 Installazione dei componenti software

2.1.1 Java Development Kit (JDK 25 LTS)

Il sistema richiede la presenza della JDK per l'esecuzione e lo sviluppo dell'applicazione. Si raccomanda l'utilizzo di una versione LTS (**Long-Term-Support**) stabile, nello specifico **JDK 25**, per garantire la piena compatibilità con i driver JDBC di PostgreSQL e il corretto supporto alle funzionalità del linguaggio.

Per procedere, è necessario collegarsi al sito ufficiale Oracle nella sezione "Java Downloads" e scaricare il pacchetto idoneo all'architettura del proprio sistema operativo (Windows, Linux e macOS).

JDK 26 JDK 25 JDK 21

Java SE Development Kit 25.0.3 downloads

JDK 25 binaries are free to use in production and free to redistribute, at no cost, under the [Oracle No-Fee Terms and Conditions \(NFTC\)](#).

JDK 25 will receive updates under the NFTC, until September 2028, a year after the release of the next LTS. Subsequent JDK 25 updates will be licensed under the [Java SE OTN License \(OTN\)](#) and production use beyond the [limited free grants](#) of the OTN license will require a fee.

Linux macOS Windows

Product/file description	File size	Download
x64 Compressed Archive	205.71 MB	https://download.oracle.com/java/25/latest/jdk-25_windows-x64_bin.zip (sha256)
x64 Installer	184.51 MB	https://download.oracle.com/java/25/latest/jdk-25_windows-x64_bin.exe (sha256)
x64 MSI Installer	183.28 MB	https://download.oracle.com/java/25/latest/jdk-25_windows-x64_bin.msi (sha256)

[Documentation Download](#)

Release Information

- [Online Documentation](#)
- [Installation Instructions](#)
- [Release Notes](#)
- [Documentation License](#)

Figura 1: Sito ufficiale Oracle per il download di Java 25

Nota per gli utenti macOS: La versione “ARM64 DMG Installer” è dedicata ai Mac con processori Apple Silicon (M1, M2, M3, ecc.), mentre la versione “x64 DMG Installer” è destinata ai Mac dotati di processore Intel.

Dopo aver completato l’installazione guidata, è necessario configurare le variabili d’ambiente per consentire l’esecuzione corretta dei comandi Java da terminale.

Windows Accedere al pannello “Variabili d’ambiente” dal menu Start. Creare una nuova variabile di sistema denominata `JAVA_HOME` che punti alla cartella radice della JDK installata (es. `C:\Program Files\Java\jdk-25`). Successivamente, selezionare la variabile `Path`, cliccare su “Modifica” e aggiungere il percorso della cartella bin: `%JAVA_HOME%\bin`.

Linux Eseguire l’estrazione dell’archivio compresso nella directory di sistema:

```
sudo mkdir -p /usr/local/java
sudo tar -xzf jdk-25_linux-x64_bin.tar.gz -C /usr/local/java/
```

Modificare il file di configurazione globale del sistema tramite il comando `sudo nano /etc/profile` aggiungendo le seguenti righe:

```
export JAVA_HOME=/usr/local/java/jdk-25
export PATH=$PATH:$JAVA_HOME/bin
```

Applicare le modifiche digitando: `source /etc/profile`.

macOS Modificare il file di configurazione della shell zsh tramite `nano ~/.zshrc` inserendo:

```
export JAVA_HOME="/Library/Java/JavaVirtualMachines/jdk-25.jdk/Contents/Home"
export PATH="$PATH:$JAVA_HOME/bin"
```

Salvare premendo `Ctrl+O`, confermare con `Invio` e uscire con `Ctrl+X`. Rendere effettive le modifiche con il comando: `source ~/.zshrc`.

Verificare la corretta installazione digitando da terminale: `java -version`.

2.2 Apache Maven

Il progetto utilizza Apache Maven per la gestione del ciclo di vita del software, l'automazione della build e la risoluzione delle dipendenze (come il driver JDBC) definite nel file `pom.xml`.



Figura 2: Sito ufficiale Apache Maven

Windows Scaricare il “Binary zip archive” dal sito ufficiale. Estrarre il contenuto in una cartella permanente (es. `C:\apache-maven-3.9.6`). Nel pannello delle variabili d'ambiente, modificare la variabile di sistema Path inserendo il percorso della cartella bin di Maven (es. `C:\apache-maven-3.9.6\bin`).

Linux Sulle distribuzioni basate su Debian/Ubuntu, l'aggiunta avviene tramite il gestore di pacchetti Apt:

```
sudo apt update
sudo apt install maven
```

macOS L'installazione consigliata avviene tramite l'ausilio del package manager Homebrew:

```
brew install maven
```

Verificare l'installazione digitando da terminale: `mvn -version`.

2.3 PostgreSQL RDBMS

Per garantire la persistenza a lungo termine dei dati relativi a utenti, film, proiezioni e prenotazioni del cinema, il sistema CineMax richiede un'istanza attiva del database relazionale PostgreSQL (versione 15 o successiva). Nell'architettura ad host singolo, il server del database viene eseguito ed ospitato localmente sulla stessa macchina su cui viene avviata l'applicazione.

2.3.1 Installazione

L'installazione e la prima inizializzazione del sistema di gestione di basi di dati variano a seconda del sistema operativo in uso:

- **Windows / macOS:** È necessario scaricare l'installer grafico dal sito ufficiale ed eseguire il setup guidato. Durante il processo, occorre definire la password per l'utente amministratore di default (**postgres**) e mantenere la porta di ascolto standard **5432**. Si raccomanda caldamente di includere l'installazione di **pgAdmin 4** (già integrato nell'installer) per usufruire di un'interfaccia grafica intuitiva atta alla gestione visuale della base dati.

- **Linux (Debian/Ubuntu):** L'installazione avviene interamente da terminale tramite il gestore di pacchetti Advanced Package Tool (**apt**) eseguendo la seguente sequenza di comandi:

```
sudo apt update
sudo apt install postgresql postgresql-contrib
sudo systemctl start postgresql
sudo systemctl enable postgresql
```

2.4 Gestione della Build e Comandi Maven

L'automazione della build, la gestione del ciclo di vita del software e la risoluzione delle dipendenze (tra cui il driver JDBC di PostgreSQL) definite nel file `pom.xml` sono interamente governate da Apache Maven.

Di seguito vengono elencati i comandi fondamentali da lanciare tramite terminale all'interno della directory radice del repository:

- **Compilazione del codice sorgente:** `mvn compile`
Effettua il parsing e la traduzione dei file sorgenti `.java` in bytecode, posizionandoli nella cartella `target/classes/`.
- **Generazione dell'eseguibile:** `mvn package`
Completa il packaging del progetto compilato, generando il file binario pronto all'uso all'interno della cartella `target/`.
- **Pulizia e build completa:** `mvn clean install`
Rimuove preventivamente la directory `target/` per eliminare i residui delle compilazioni precedenti e riesegue l'intero ciclo di build da zero, garantendo la coerenza logica del codice sorgente.
- **Generazione automatica delle JavaDoc:** `mvn javadoc:javadoc`
Analizza i commenti strutturati nel codice sorgente ed esporta la documentazione tecnica completa in formato HTML direttamente all'interno della cartella `doc/javadoc/`.

2.5 Ambienti di Sviluppo Integrati (IDE)

Lo sviluppo, il refactoring e il debugging del codice sorgente sono stati condotti mediante l'ausilio dei seguenti ambienti di sviluppo integrati:

- **IntelliJ IDEA** (Community o Ultimate Edition)
- **Eclipse IDE** for Enterprise Java Developers

Il progetto può essere importato all'interno dell'IDE di preferenza selezionando l'opzione **“Open”** o **“Import Project”** e puntando direttamente al file descrittore `pom.xml` situato nella directory radice del repository. Questa procedura consente all'IDE di identificare correttamente la struttura del software e di scaricare autonomamente dal Maven Central Repository tutte le librerie esterne richieste, inclusi i driver JDBC per il collegamento a PostgreSQL.

3 Progettazione e architettura del software (UML)

La transizione da un sistema monolitico a una soluzione distribuita richiede una rigorosa pianificazione strutturale, volta a massimizzare i requisiti non funzionali di scalabilità, manutenibilità, estendibilità e sicurezza. In questo capitolo viene analizzata la scomposizione logica del sistema **CineMax**, dettagliando le scelte architetturali adottate, l'organizzazione dei package in termini di coesione e accoppiamento, i design pattern applicati e la modellazione dinamica dei flussi critici concorrenti.

3.1 Modello Architeturale

Il sistema **CineMax** adotta un modello architeturale di tipo **Three-Tier (architettura a tre livelli)** innestato su un paradigma distribuito **Client/Server** mediante l'infrastruttura di comunicazione Java RMI. Questa topologia consente una netta separazione delle responsabilità (*Separation of concerns*), isolando l'interazione con l'utente, le regole di business e la persistenza dei dati in layer logici indipendenti e fortemente disaccoppiati.

Il partizionamento dei tre livelli è strutturato come segue:

1. **Presentation Tier (Livello di Presentazione):** Allocato interamente sul modulo `clientCM`, è costituito dall'interfaccia grafica (GUI). Questo livello ha la responsabilità esclusiva di renderizzare le informazioni a schermo, catturare gli input dell'utente (Ospiti, Clienti, Proiezionisti, Bigliettai) ed eseguire una prima validazione sintattica locale (es. controllo dei campi vuoti o formati stringhe). Non possiede alcuna logica di business né accesso diretto al database; ogni operazione dispositiva viene delegata al livello inferiore tramite l'invocazione di metodi remoti esposti dagli stub RMI.
2. **Application Business Logic Tier (Livello Logico):** Residente sul modulo `serverCM`, rappresenta il core operativo del sistema. Questo livello implementa i vincoli di integrità semantica della sala cinematografica (ad esempio, il controllo della sovrapposizione temporale dei palinsesti o la verifica della disponibilità effettiva di un posto prima della transazione). Il server mappa le richieste RMI entranti su thread concorrenti, orchestrando l'accesso sicuro alle risorse condivise mediante primitive di sincronizzazione avanzate, prevenendo attivamente fenomeni di *race condition*.
3. **Data Tier (Livello Dati):** Costituito dal DBMS relazionale PostgreSQL. Questo tier è deputato alla memorizzazione permanente, alla consistenza transazionale (proprietà ACID) e alla sicurezza dei dati a lungo termine. Il Business Logic Tier interagisce con questo livello esclusivamente tramite l'interfaccia standard JDBC, isolando le query SQL all'interno di componenti specializzati.

Grazie a questa architettura, modifiche strutturali alla GUI non impattano sulla logica computazionale, così come un cambio di schema nel database o una migrazione ad un altro DBMS (es. da PostgreSQL a Oracle) richiede modifiche circoscritte unicamente al Data Tier, preservando intatto il resto del software.

3.2 Diagramma delle classi e organizzazione dei package

La struttura interna del codice segue una scomposizione in sotto-package guidata dal principio della massima coesione interna e del minimo accoppiamento esterno. Il package radice `cinemax` è ripartito in tre macro-aree logiche fondamentali: **common**, **client** e **server**.



Figura 3: Diagramma delle Classi UML

Il diagramma sopra illustrato mostra la modellazione orientata agli oggetti delle entità di dominio presenti nel progetto nel sotto-package `common.model`. Classi fondamentali come `Utente`, `Film`, `Proiezione` e `Prenotazione` implementano l'interfaccia `Serializable` per consentire il passaggio per valore attraverso la rete. Si evidenziano attributi critici come la cifratura delle credenziali (`passwordCifrata`), la gestione delle capienze (`postiLiberi`) e i vincoli d'età delle pellicole (`etaMinima`).

Di seguito viene esposta la scomposizione analitica dei package e dei relativi componenti software:

3.2.1 Package `cinemax.common`

Rappresenta il punto di contatto logico e strutturale condiviso obbligatoriamente sia dal client che dal server. Contiene le interfacce remote RMI e gli oggetti di trasferimento dati (**DTO** / **Model**) che viaggiano sulla rete.

- **CinemaService (Interfaccia):** Estende `java.rmi.Remote`. Dichiarerà tutti i metodi che il server si impegna ad esporre remoti (es. `autenticaUtente()`, `getPrenotazioniAttive()`, `prenotaPosto()`). Ogni metodo lancia un'eccezione controllata `RemoteException`.
- **Classi di Modello (Utente, Film, Proiezione, Prenotazione):** Sono classi Plain Old Java Objects (POJO) pure, implementanti l'interfaccia standard `java.io.Serializable`.

La serializzazione è un requisito vincolante di Java RMI per consentire il passaggio di parametri e valori di ritorno per valore (*pass-by-value*) attraverso i socket di rete, convertendo lo stato delle istanze in flussi di byte.

3.2.2 Package `cinemax.client`

Contiene l'infrastruttura legata alla presentazione visiva e alla cattura degli eventi.

- **ClientMain:** Punto di ingresso dell'applicazione client; esegue la ricerca (`lookup`) nel registro RMI per ottenere il riferimento remoto dello stub di `CinemaService` e avvia l'interfaccia grafica.
- **Sotto-package `.view`:** Comprende le classi grafiche responsabili del layout visivo (es. `LoginFrame`, `DashboardClient`, `PalinsestoPanel`, `MappaSalaComponent`).
- **Sotto-package `.controller`:** Classi mediatrici che intercettano le azioni dell'utente sulla GUI (es. la pressione di un pulsante) e invocano i metodi corrispondenti sullo stub RMI, aggiornando successivamente la vista in base alla risposta ricevuta.

3.2.3 Package `cinemax.server`

Contiene l'implementazione della logica di business e governa l'accesso ai servizi di backend.

- **ServerMain:** Inizializza l'istanza del server, istanzia il registro RMI locale e vi effettua il `rebind` dell'oggetto remoto sotto una stringa identificativa univoca.
- **CinemaServiceImpl:** Estende `UnicastRemoteObject` e realizza concretamente l'interfaccia remota `CinemaService`. Al suo interno avviene il coordinamento della logica applicativa, la validazione dei vincoli aziendali e l'invocazione selettiva dei layer di persistenza.
- **Sotto-package `.dao (Data Access Object)`:** Isola la logica di persistenza. Contiene le interfacce e le implementazioni JDBC concrete (es. `UtenteDAO`, `UtenteDAOImpl`, `ProiezioneDAOImpl`) incaricate di dialogare direttamente con il database PostgreSQL tramite query SQL native strutturate.

3.3 Design patterns utilizzati

Al fine di garantire flessibilità strutturale, riuso del codice e aderenza ai sani principi della programmazione orientata agli oggetti (SOLID), sono stati implementati in modo mirato i seguenti design pattern formali:

3.3.1 Singleton Pattern (Creazionale)

Applicato alla classe `DatabaseConnectionManager` presente nel modulo server. L'apertura e la chiusura incontrollata di connessioni di rete verso un DBMS è un'operazione computazionalmente onerosa che può portare al rapido esaurimento delle risorse del server.

- **Giustificazione:** Il pattern garantisce l'esistenza di un'unica istanza centralizzata del manager di connessione per l'intera Java Virtual Machine del server. Fornisce un punto di accesso globale (`getInstance()`) e incapsula i parametri di configurazione del driver JDBC (URL, username, password), ottimizzando la gestione dei canali di comunicazione verso PostgreSQL.

3.3.2 Data Access Object - DAO Pattern (Strutturale)

Utilizzato capillarmente per mediare la persistenza di tutte le entità del dominio (*Utente*, *Proiezione*, *Prenotazione*, *Film*).

- **Giustificazione:** Consente di disaccoppiare radicalmente il layer della logica di business (*CinemaServiceImpl*) dal codice SQL specifico di PostgreSQL. La logica applicativa invoca metodi astratti descritti dalle interfacce (es. `proiezioneDao.findOverlappingShowtimes()`), rimanendo completamente agnostica su come i dati siano memorizzati (tabelle relazionali, file XML, database NoSQL), incrementando l'indice di manutenibilità del software.

3.3.3 Factory Pattern (Creazionale)

Abbinato al pattern DAO tramite la classe *DAOFactory*.

- **Giustificazione:** Centralizza e scherma il processo di istanziazione dei componenti di accesso ai dati. Il server richiede le istanze dei DAO direttamente alla Factory. Se in futuro si rendesse necessario introdurre un meccanismo di caching o mutare l'implementazione concreta del DAO, sarà sufficiente intervenire sulla Factory senza alterare le classi client che ne fanno uso.

3.3.4 Observer Pattern (Comportamentale)

Implementato per abilitare l'aggiornamento dinamico delle interfacce grafiche a seguito di mutazioni dello stato del server (architettura basata su eventi). Nel contesto distribuito Java RMI, assume la forma di **Remote Observer Pattern**.

- **Giustificazione:** Quando un cliente prenota con successo uno o più posti all'interno di una determinata proiezione, la variazione di disponibilità deve riflettersi istantaneamente sulle GUI di tutti gli altri client sintonizzati sulla medesima schermata, per prevenire tentativi di acquisto su posti ormai occupati. I client registrano un proprio stub di ascolto remoto presso il server; al verificarsi della transazione, il server itera sulla lista dei listener attivi invocando in modalità push il metodo remoto di notifica (`updateView()`), forzando il ridisegno della mappa dei posti sul client.

3.4 Processo transazionale e concorrente di prenotazione posto

Il flusso critico del sistema riguarda la prenotazione simultanea dello stesso identico posto (es. Riga 5, Colonna 10) per una determinata proiezione da parte di due utenti concorrenti ad un millisecondo di distanza. Per scongiurare fenomeni di *overbooking* o stati inconsistenti della base dati, il flusso dinamico applica logiche di mutua esclusione sia a livello software che a livello DBMS.

Di seguito viene esposta la sequenza logica dettagliata delle operazioni (che mappa in modo speculare il relativo Diagramma di Sequenza UML):

1. **Invocazione remota:** Il *ClientController* cattura l'evento di click sul posto della mappa e invoca il metodo remoto dello stub: `cinemaService.prenotaPosto(idProiezione, riga, colonna, idUtente)`.
2. **Thread allocation:** La JVM del Server intercetta la richiesta di rete e alloca un thread dedicato dal pool RMI per servire la chiamata all'interno di *CinemaServiceImpl*.

3. **Inizio transazione SQL:** Il server richiede una connessione al `DatabaseConnectionManager`. Al fine di controllare accuratamente le singole operazioni, disabilita il commit automatico di JDBC (`connection.setAutoCommit(false)`), avviando formalmente una transazione ACID.

4. **Lock di lettura (Pessimistic Locking):** Il thread esegue una query SQL tramite il `PrenotazioneDAO` sul posto selezionato, sfruttando la clausola `FOR UPDATE` di PostgreSQL:

```
SELECT stato FROM posto WHERE id_proiezione = ?
AND riga = ? AND colonna = ? FOR UPDATE;
```

Il DBMS PostgreSQL blocca la riga corrispondente nel database. Se un secondo thread concorrente tenta di verificare lo stesso posto, viene posto in stato di attesa (*blocked*) finché la prima transazione non si conclude.

5. **Verifica semantica:** Il Business Logic Tier riceve il risultato della query. Se lo stato del posto è `LIBERO`, il flusso procede. Se lo stato risulta già `OCCUPATO` (poiché il thread concorrente è arrivato un istante prima), viene sollevata un'eccezione personalizzata (`PostoGiaPrenotatoException`), la transazione esegue il `rollback` e il client riceve un messaggio di errore visivo sulla GUI.

6. **Scrittura e generazione codice:** Il server genera un codice di prenotazione alfanumerico univoco, inserisce il record nella tabella `prenotazione` ed esegue l'update dello stato del posto a `OCCUPATO`.

7. **Commit e Notifica dei Remote Observers:** Viene invocato il metodo `connection.commit()` per rendere permanenti e visibili le modifiche a livello globale nel database, rilasciando contestualmente il lock sulla riga. Successivamente, il server avvia un thread asincrono per scorrere la lista dei `RemoteObservers` registrati e notificare la variazione a tutti i client attivi, affinché la cella sulla mappa si colori di rosso in tempo reale.

8. **Risposta al client:** Il metodo remoto si chiude restituendo l'oggetto `Prenotazione` serializzato al client originario, il quale mostra la schermata di conferma con il codice univoco generato.

3.4.1 Fase di autenticazione e login

La fase di login descrive la transizione di stato del sistema da un utente anonimo (*Ospite*) a un utente profilato e dotato di autorizzazioni specifiche basate su ruoli (RBAC).

Il diagramma di attività e sequenza per questa fase si articola nei seguenti passi essenziali:

1. L'utente inserisce le proprie credenziali (`username` e `password` in testo chiaro) nei campi di testo della classe `LoginFrame` e clicca sul pulsante di accesso.
2. Il controller del client intercetta l'action, estrae le stringhe e invoca il metodo remoto `cinemaService.autenticaUtente(username, password)`.
3. Il server riceve le stringhe. Per ovvie ragioni di sicurezza e conformità ai principi di crittografia, le password non sono memorizzate in chiaro nel database. Il server calcola il valore di hash della password inserita utilizzando un algoritmo di hashing sicuro (es. SHA-256) combinato con una stringa di *salt* univoca.

4. Il server interroga il `UtenteDAO` passando l'username e l'hash calcolato. Il DAO esegue una query mirata sulla tabella `utente`.
5. Se non viene trovata alcuna corrispondenza, il DAO restituisce un valore nullo; il server interpreta il risultato negativo e risponde lanciando una `CredenzialiInvalideException`, inducendo la GUI del client a mostrare una notifica pop-up di errore in rosso senza alterare lo stato della visualizzazione attuale.
6. Se la corrispondenza ha successo, il database restituisce le informazioni anagrafiche dell'utente e il suo ruolo associato all'interno del sistema (`CLIENTE`, `PROIEZIONISTA`, `BIGLIETTAIO`).
7. Il server istanzia un oggetto DTO di tipo `Utente`, lo popola con i dati estratti (escludendo l'hash della password per ragioni di sicurezza nel transito di rete) e lo restituisce al client.
8. Il client riceve l'oggetto `Utente`. La classe `ClientMain` distrugge l'istanza di `LoginFrame` per liberare memoria locale e, analizzando il campo `ruolo` contenuto nel DTO, istanzia dinamicamente la specifica dashboard autorizzativa (es. abilita i menu di configurazione palinsesto solo ed esclusivamente se il ruolo corrisponde a `PROIEZIONISTA`), completando la segregazione delle responsabilità a runtime.

4 Progettazione della base di dati ER Relazionale

In questo capitolo viene dettagliato l'intero processo di progettazione, strutturazione e implementazione della base di dati relazionale a supporto del sistema *CineMax*. L'obiettivo primario è definire un'architettura dei dati persistente, robusta e coerente, capace di gestire in modo efficiente le limitazioni relative agli utenti (clienti e personale), alla programmazione delle proiezioni cinematografiche e alla gestione delle prenotazioni dei posti in sala.

Il percorso metodologico segue i canoni standard dell'ingegneria dei dati e si articola in più fasi sequenziali: si parte da un'analisi e purificazione dei requisiti per definire il dizionario dei dati, si formalizza il Modello concettuale mediante il diagramma Entità-Relazionale (ER), si analizzano le regole di business non esprimibili graficamente e si procede alla ristrutturazione dello schema eliminando i costrutti complessi. Infine, viene presentata la traduzione nel Modello logico relazionale, supportata da una verifica formale delle forme normali (BCNF/3FN) per garantire l'assenza di anomalie di aggiornamento, concludendo con le specifiche tecniche sul popolamento iniziale della base di dati tramite script dedicati.

4.1 Analisi dei requisiti

Il sistema *CineMax* richiede la gestione persistente dei dati relativi a un'infrastruttura cinematografica. Il sistema prevede tre tipologie di utenti registrati: Clienti, Bigliettai e Proiezionisti. Ogni utente è identificato univocamente dalla propria email e possiede un nome, un cognome e una password. Il cinema programma una serie di proiezioni. Ogni proiezione si riferisce a uno specifico film (caratterizzato da titolo, regista, genere e durata), avviene in una determinata sala in una specifica data e ora, possiede un prezzo di biglietto e tiene traccia dei posti attualmente disponibili. I clienti possono effettuare prenotazioni per una determinata proiezione, specificando il numero di posti desiderati. Il sistema deve garantire la coerenza dei posti della sala e impedire sovrapprenotazioni.

4.2 Modello concettuale (Diagramma ER)

Il diagramma concettuale esprime le entità principali del sistema e le relazioni cardinali che le governano, coerentemente con i modelli definiti nel sistema:

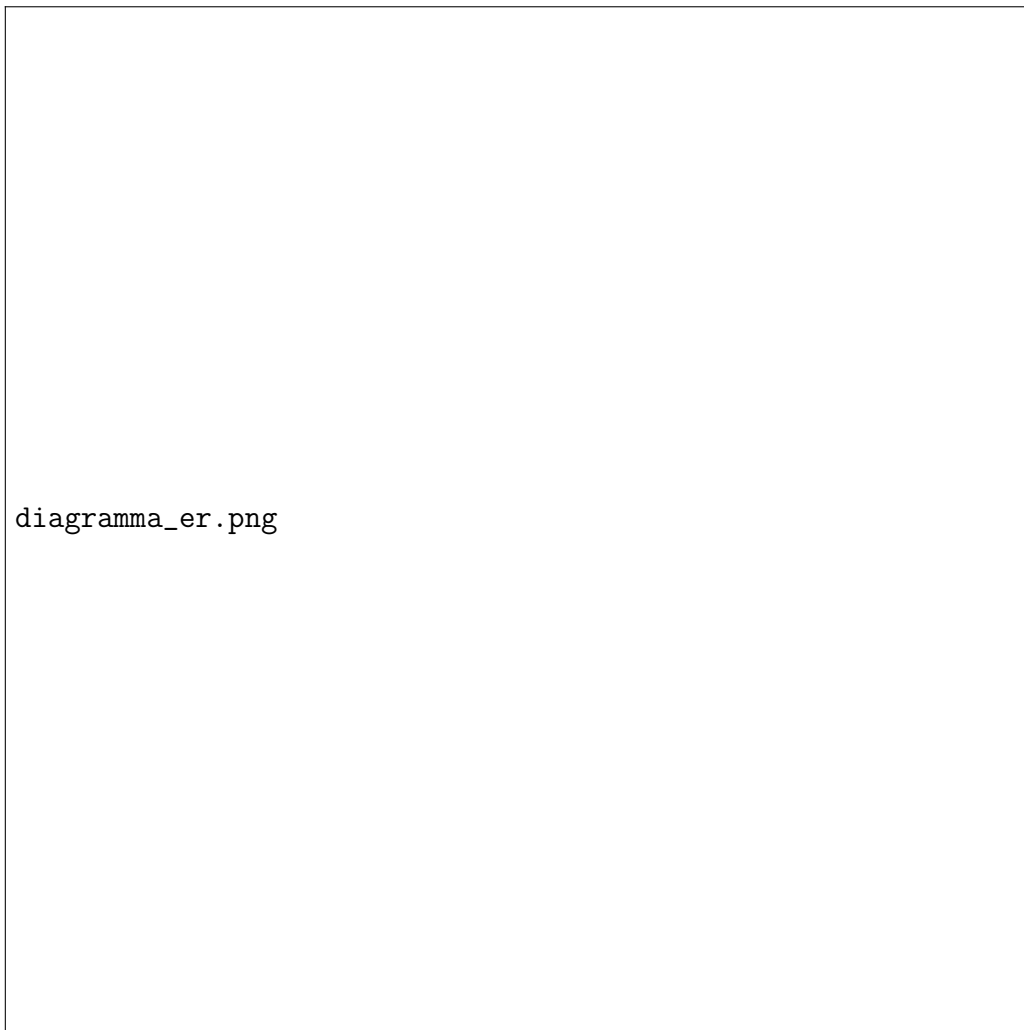


Figura 4: Diagramma Entità-Relazione (ER) del Database CineMax

4.2.1 Specifiche delle relazioni e cardinalità

- **Effettua:** Associazione tra *Utente* e *Prenotazione*. Un Utente (con ruolo Cliente) può effettuare da 0 a N prenotazioni; una Prenotazione appartiene a uno e un solo Utente. (*Cardinalità: 1 a N*).
- **Riferita a:** Associazione tra *Prenotazione* e *Proiezione*. Una Prenotazione è associata a una e una sola Proiezione; una Proiezione può ricevere da 0 a N prenotazioni. (*Cardinalità: 1 a N*).
- **Caratterizza:** Associazione tra *Film* e *Proiezione*. Un Film può essere programmato in 0 o più Proiezioni; una Proiezione si riferisce a uno e un solo Film. (*Cardinalità: 1 a N*).

4.3 Vincoli in linguaggio naturale

I seguenti vincoli di integrità e regole di business non sono direttamente esprimibili mediante le notazioni standard del diagramma ER e vengono implementati a livello applicativo o tramite vincoli CHECK sul DBMS PostgreSQL:

1. **Coerenza temporale proiezioni:** La data e l'ora di inserimento di una nuova proiezione devono essere strettamente successive alla data odierna.

2. **Vincolo di capacità:** Il valore `num_posti` inserito in una singola prenotazione deve essere minore o uguale ai `posti_disponibili` della relativa proiezione al momento della transazione.
3. **Autorizzazione ruoli (RBAC):** Solamente gli utenti aventi valore `ruolo = 'CLIENTE'` possono essere associati come intestatari di una riga nella tabella delle prenotazioni. Solamente gli utenti con `ruolo = 'PROIEZIONISTA'` possono eseguire operazioni di inserimento o modifica sulle proiezioni.
4. **Unicità di sala:** Non possono essere programmate due proiezioni nella stessa sala alla stessa data la cui distanza temporale sia inferiore alla durata del film associato alla prima proiezione aumentata di un intervallo di pulizia della sala standard (30 minuti).

4.4 Ristrutturazione dello schema ER

4.4.1 Eliminazione di attributi composti o multivalore

- **Ruolo:** L'attributo ruolo dell'utente è modellato come un tipo enumerativo atomico (`Ruolo`), garantendo l'assenza di strutture composte direttamente all'interno dell'entità.
- **Dati anagrafici:** Nome e cognome sono stati separati fin dal principio come attributi atomici testuali distinti per evitare stringhe composte difficili da indicizzare e manipolare.

4.4.2 Analisi delle ridondanze

- **Attributo `posti_disponibili` in *Proiezione*:** Dal punto di vista puramente teorico, questo attributo rappresenta una ridondanza, in quanto il numero di posti disponibili potrebbe essere calcolato dinamicamente a runtime effettuando una sottrazione tra la capacità nominale della sala (200 posti) e la sommatoria (SUM) dell'attributo `num_posti` di tutte le prenotazioni associate a quella specifica proiezione.
- **Motivazione del mantenimento:** Si è scelto formalmente di **mantenere la ridondanza** per ragioni di ottimizzazione prestazionale delle query di lettura frequenti sulla dashboard del client, diminuendo il carico computazionale sul DBMS ed evitando join onerosi sul calcolo in tempo reale.

4.5 Modello Logico Relazionale

Si procede alla traduzione dello schema E-R ristrutturato nel modello logico relazionale. Le chiavi primarie sono indicate in sottolineato, mentre le chiavi esterne sono contrassegnate dalla sigla FK.

- **UTENTI** (username, nome, cognome, password_hash, data_nascita, domicilio, ruolo)
- **FILM** (id_film, titolo, genere, regista, anno, durata_minuti, eta_minima)
- **PROIEZIONI** (id_proiezione, data_ora, *id_film*, prezzo_biglietto)
- **PRENOTAZIONI** (codice_univoco, *username_cliente*, *id_proiezione*, numero_posti, data_creazione)

4.6 Analisi delle Dipendenze Funzionali e Normalizzazione

Al fine di garantire l'assenza di anomalie di inserimento, cancellazione e aggiornamento, le relazioni strutturate sono state sottoposte a verifica formale rispetto alla **Terza Forma Normale (3FN)** e alla **Forma Normale di Boyce-Codd (BCNF)**.

Prendendo in esame la relazione **UTENTI**:

username → nome, cognome, password_hash, data_nascita, domicilio, ruolo

L'unica dipendenza funzionale ha come determinante la chiave primaria (username), che è superchiave. Non esistono dipendenze transitive tra attributi non chiave. Pertanto, la relazione soddisfa pienamente i requisiti della BCNF. Lo stesso principio di scomposizione atomica è stato applicato alle restanti tabelle, garantendo che ogni determinante sia una superchiave.

4.7 Script DDL di Creazione del Database (PostgreSQL)

Di seguito si riporta lo script SQL nativo per la generazione delle tabelle e l'applicazione dei vincoli di integrità referenziale e semantica all'interno del RDBMS:

```
-- Creazione del tipo enumerativo richiesto dalla tabella utenti
CREATE TYPE ruolo_enum AS ENUM ('CLIENTE', 'BIGLIETTAIO', '
    ↳ PROIEZIONISTA');

CREATE TABLE utenti (
    username          VARCHAR(50)    PRIMARY KEY,
    nome              VARCHAR(50)    NOT NULL,
    cognome           VARCHAR(50)    NOT NULL,
    password_hash     VARCHAR(255)   NOT NULL,
    data_nascita      DATE,
    domicilio         VARCHAR(150)   NOT NULL,
    ruolo             ruolo_enum     NOT NULL
);

CREATE TABLE film (
    id_film           SERIAL          PRIMARY KEY,
    titolo            VARCHAR(100)    NOT NULL,
    genere            VARCHAR(100)    NOT NULL,
    regista           VARCHAR(100)    NOT NULL,
    anno              SMALLINT        NOT NULL CHECK (anno BETWEEN 1888
    ↳ AND 2100),
    durata_minuti     SMALLINT        NOT NULL CHECK (durata_minuti > 0),
    eta_minima        SMALLINT        NOT NULL DEFAULT 0 CHECK (
    ↳ eta_minima >= 0)
);

CREATE TABLE proiezioni (
    id_proiezione     SERIAL          PRIMARY KEY,
    data_ora          TIMESTAMP       NOT NULL,
    id_film           INT             NOT NULL REFERENCES film(id_film)
    ↳ ON DELETE CASCADE,
    prezzo_biglietto NUMERIC(6,2)    NOT NULL CHECK (prezzo_biglietto >=
    ↳ 0)
);
```

```
CREATE TABLE prenotazioni (  
    codice_univoco      VARCHAR(36)  PRIMARY KEY,  
    username_cliente    VARCHAR(50)  NOT NULL REFERENCES utenti(  
        ↳ username) ON DELETE CASCADE,  
    id_proiezione       INT          NOT NULL REFERENCES proiezioni(  
        ↳ id_proiezione) ON DELETE CASCADE,  
    numero_posti        SMALLINT     NOT NULL CHECK (numero_posti > 0)  
        ↳ ,  
    data_creazione      TIMESTAMP    NOT NULL DEFAULT NOW()  
);
```

5 Sitografia e bibliografia

- Oracle Java Documentation <https://docs.oracle.com/en/java/>
- Apache Maven Project <https://maven.apache.org/>
- PostgreSQL Official Documentation: <https://www.postgresql.org/docs/>